

1. [5 pts.] Study and understand the "greedy-choice property" (pages 427-428). The greedy choice, if locally optimal (possibly, but not necessarily, based on past choices), should lead to a globally optimum solution, independent of future choices. How does the fractional knapsack problem satisfy the greedy-choice property?

The Fractional Knapsack problem lets you take pieces of each material rather than taking all of each choice. This lets you go with the greedy solution of always selecting the most valuable (or value dense) material FIRST since you can take a portion of any material and when you are at the full point you can take a portion of what's left.

Ex
All gold + All Silver + A portion of Bronze

2. [5 pts.] What is an optimal Huffman code for each of the following symbols, based on their frequencies?
a:10 b:2 c:4 d:5 e:15 f:9 g:6 h:8 i:12 j:1
(Although you may, you won't necessarily have to convert these frequencies into percentages!)

$$10 + 2 + 4 + 5 + 15 + 9 + 6 + 8 + 12 + 1 = 72$$

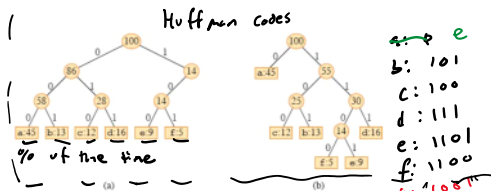
$$e = 20.5\% \quad h = 11.1\% \quad b = 2.7\%$$

$$i = 16.6\% \quad g = 8.3\% \quad j = 1.3\%$$

$$a = 13.8\% \quad d = 6.9\% \quad c = 5.5\%$$

$$f = 12.5\%$$

In greedy Algo notes



$$j + b = 3$$

$$3 + c = 7$$

$$d + g = 11$$

$$4 + 5 = 19$$

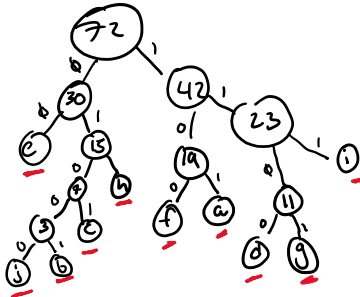
$$f + a = 19$$

$$11 + i = 23$$

$$c + 19 = 30$$

$$7 + 23 = 42$$

$$30 + 42 = 72$$



$$e = 100$$

$$j = 10000$$

$$b = 10001$$

$$c = 1001$$

$$h = 1011$$

$$f = 1000$$

$$a = 1001$$

$$d = 1100$$

$$g = 1101$$

$$i = 111$$

W.

3. [5 pts.]

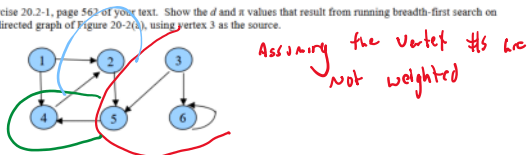
(a) Exercise 20-2-1, page 563 of your text. Show the d and n values that result from running breadth-first search on the directed graph of Figure 20-2(a), using vertex 3 as the source.



Assuming the vertex #s are 1, 2, 3

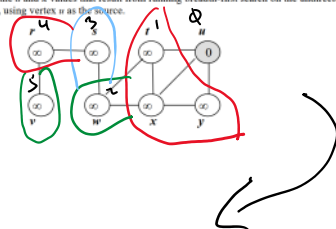
3. [5 pts.]

(a) Exercise 20.2-1, page 563 of your text. Show the d and π values that result from running breadth-first search on the directed graph of Figure 20-2(a), using vertex 3 as the source.



Vertex	d	π
1	∞	NIL
2	3	4
3	0	NIL
4	2	5
5	1	3
6	1	3

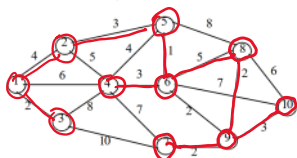
(b) Show the d and π values that result from running breadth-first search on the undirected graph of the following figure, using vertex u as the source.



Vertex	d	π
u	0	NIL
v	1	u
w	1	u
x	1	u
y	2	x
z	3	y
t	4	z
s	5	t

Vertex	d	π
u	0	NIL
v	1	u
w	1	u
x	1	u
y	2	x
z	3	y
t	4	z
s	5	t

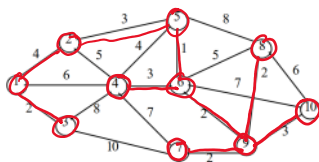
4. [10 pts.] Find the Minimum Spanning Tree for a graph $G = (V, E)$ using each of (a) *Prim's* algorithm and (b) *Kruskal's* algorithm as described in class. Show your work.



a) Prim's

Vertex	Key	START	(1)	(3)	(2)	(5)	(6)	(9)	(7)	(8)	(10)	(4)
1	NIL	0/NIL										
2	NIL		4(1)									
3	NIL		2(1)									
4	NIL			6(3)	5(2)	4(5)	3(6)					
5	NIL				3(2)							
6	NIL					1(5)						
7	NIL							2(9)				
8	NIL							5(8)	2(7)			
9	NIL							2(8)				
10	NIL							7(6)	3(9)			

b) Kruskal



$(5,6)$: safe

$(7,9)$: safe

$(1,2)$: safe

$(8,9)$: safe

$(1, 2)$: safe

$(8, 9)$: safe

$(6, 9)$: safe

$(7, 5)$: safe

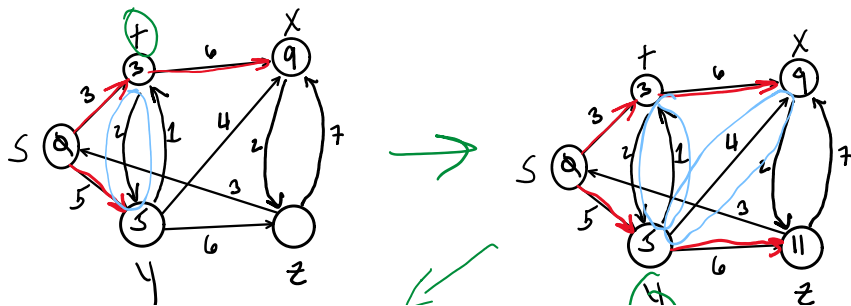
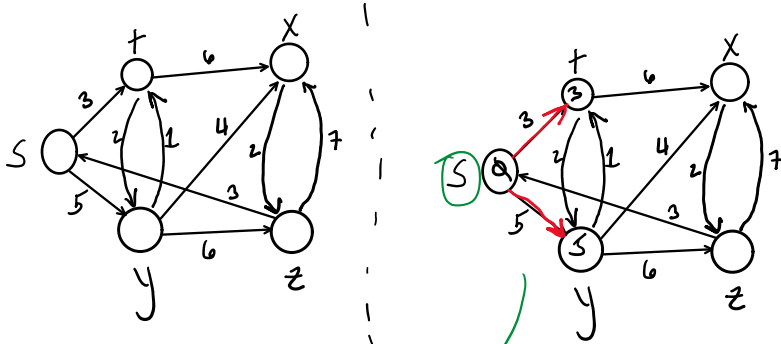
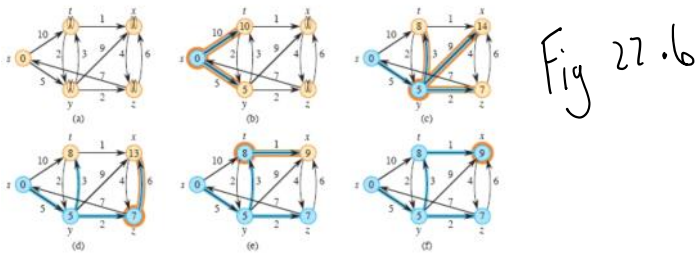
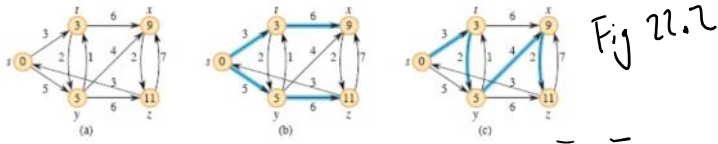
$(9, 10)$: safe

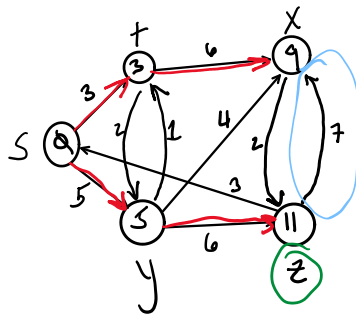
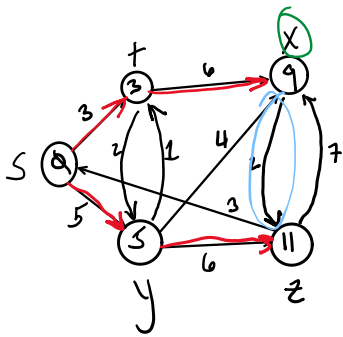
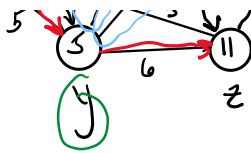
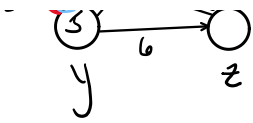
$(4, 6)$: safe

$(1, 2)$: safe

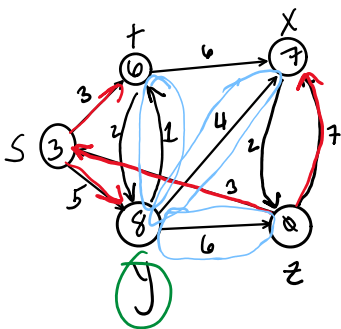
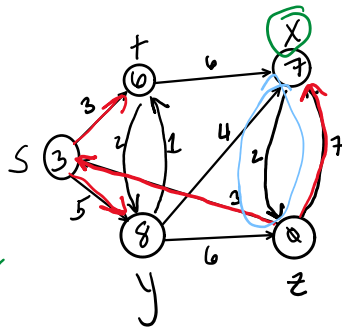
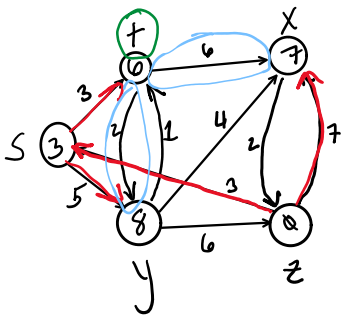
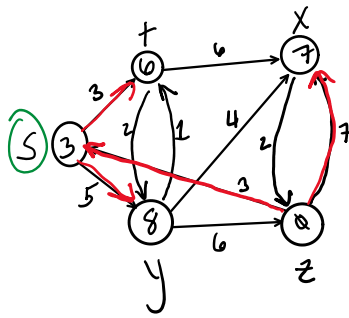
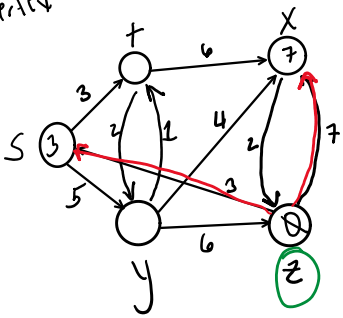
All points connected!

5. [5 pts.] Exercise 22.3-1, page 624 of your text. Run Dijkstra's algorithm on the directed graph of Figure 22.2, first using vertex s as the source and then using vertex z as the source. In the style of Figure 22.6, or some other good way you may choose, show the d and π values and the vertices in set S after each iteration of the **while** loop.





Vertex Z



6. [0 pts.] Exercise 22.3-3, page 624 of your text. Suppose we change line 6 of Dijkstra's algorithm to the following:
 while $|Q| > 1$

This change causes the "while" loop to execute $|V|-1$ times instead of $|V|$ times. Is this proposed algorithm correct?

No, it wouldn't go to the last element so the "relaxing" wouldn't happen. Other connected vertices also wouldn't be updated (if relevant).

DIJKSTRA'S ALGORITHM (continued)

```

DIJKSTRA( $G, w, s$ )
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = \emptyset$ 
4  for each vertex  $u \in G.V$ 
5      INSERT( $Q, u$ )
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8       $S = S \cup \{u\}$ 
9      for each vertex  $v$  in  $G.Adj[u]$ 
10         RELAX( $u, v, w$ )
11         if the call of RELAX decreased  $v.d$ 
12             DECREASE-KEY( $Q, v, v.d$ )
  
```

S is set of vertices of which we found the shortest path.

Always stuff up in Priority Queue (heap)